

Ch3

3-1

DOM

When a web browser loads an HTML page from a web server, it **converts the HTML code into a tree** data structure known as **document tree**. All user interactions in the client-side of a web app are done through this DOM tree.

The Document Object Model standard ([online reference](#)) defines a programming interface to access the document tree. Each HTML element is represented by a **node** in the tree. DOM defines the properties, methods and events for these nodes, and the nodes are accessible to JavaScript as objects. In summary, **DOM defines how JavaScript code can access and manipulate nodes in a document tree**.

DOM --> MVVM

DOM v.s. MVVM => 自动绑定 vs 手动绑定

DOM : 数据变了, 手动更新DOM

JavaScript |

```
1 let username = "Congwen";
2 document.getElementById("nameDisplay").innerText = username;
3 username = "Evelyn"; //更改数据
4 document.getElementById("nameDisplay").innerText = username;
5 //更改完数据之后, 需要手动告知页面 -> 告知数据更改, 请找新数据
```

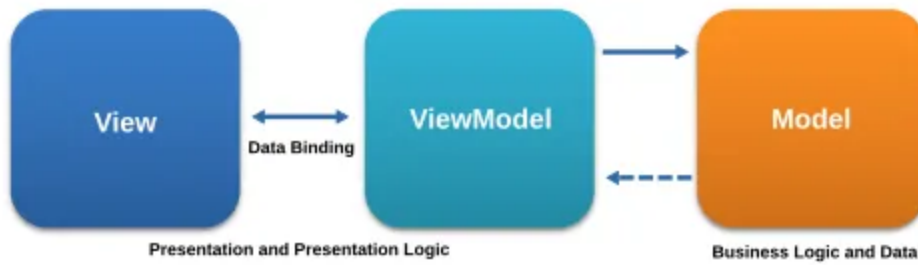
Vue 的响应式系统 (自动同步)

Vue |

```
1 data() {
2   return {
3     username: "Congwen"
4   };
5 }
6
7 this.username = "Evelyn"; //更改数据, 不需要告知 -- 自动同步
```

更新由框架负责 Vue updates the view efficiently.

MVVM



Model – Data layer → Raw data

View – The structure, layout, and appearance of what a user sees on the screen.

ViewModel → Data processing

ViewModel 是一个**中介层**，它的职责是：

- **桥接 Model 和 View**：将 Model 中的原始数据转换成 View 可用的格式
- **处理界面逻辑**：如按钮状态、表单验证、用户输入响应等
- **实现双向绑定**：View 的变化自动更新 Model，Model 的变化自动更新 View

数据 (Model) ≠ 处理数据的逻辑 (ViewModel)，而界面 (View) 又不应该承担数据处理职责，所以需要**一个中介层——ViewModel**。

JavaScript |

```
1 export default {
2   data() {
3     return {
4       username: "Congwen" //Model
5     };
6   },
7   computed: {
8     greeting() {
9       return `Hello, ${this.username}`; //ViewModel
10    }
11  },
12  methods: {
13    updateName(newName) {
14      this.username = newName;
15    }
16  },
17  template: `<p>{{ greeting }}</p>` // VIEW 界面
18 }
```

Loading Vue.js

1. CDN 加载 Vue --> 直接在网页中引入Vue的核心库

直接在 HTML 文件中写 JavaScript，导入Vue

HTML

```
1 <script src="https://unpkg.com/vue@3"></script>
```

把这段 `<script>` 标签放在 HTML 文件中，就可以使用 Vue 的功能了。

- `unpkg.com` 是一个公共 CDN，可以快速加载开源库。
- `vue@3` 表示加载 Vue 的第 3 版。
- 简单快速体验功能，不合适大项目
- 在线加载 Vue 核心库
- 没有结构，纯 HTML

2. npm init @vue --> 创建完整 Vue 项目结构

- 本地安装 Vue 包
- 初始化一个Vue项目模板
 - 自动创建项目文件夹和结构（如 `src`，`public`，`package.json`）
 - 安装 Vue 相关依赖（如 `vue`，`vite`，`eslint`，`vue-router`，`pinia` 等）

Node.js & NPM

Node.js 是一个运行 JavaScript 的服务器环境，它的核心作用是：

- 让你在本地或服务器上运行 JS（不依赖浏览器）
- 提供开发工具（比如构建、打包、测试）
- 支持包管理系统（npm）

NPM（Node Package Manager）是 Node.js 附带的工具，它可以：

- 下载并安装开源库（比如 Vue、React、Axios 等）
- 管理依赖版本
- 组织项目结构（通过 `package.json`）

NPM 是一个工具，它本身并不“内含”Vue，而是提供了一个机制来安装和管理 Vue 这样的库。

- npm（Node Package Manager）是一个包管理工具，它连接到一个远程仓库（npm registry），里面存放了成千上万的 JavaScript 库，包括 Vue。
- 当你执行 `npm install vue` 时，npm 会去 registry 下载 Vue 的源码包，并把它放到你的项目中。

所以 Vue 是一个独立的 npm 包，而 npm 是安装它的工具

3-2

Minimal Vue Application

```
<div id="app">
  <div>1 + 2 * 3 = {{ 1 + 2 * 3 }}</div>
  <div>It's now {{ (new Date).toLocaleTimeString() }}</div>
</div>

<script>
// define an empty Vue options object
const options = { };

// create a Vue component, mounted to the HTML template
const vm = Vue.createApp(options).mount("#app");
</script>
```

A Vue application is created using `Vue.createApp()` . 返回一个应用对象

Vue 应用必须挂载到一个 DOM 元素上，通常是 `<div id="app"></div>` 。

In the **template**, you can use the moustache `{{ }}` notation to run JavaScript expressions.

The result of the expression will be inserted into the template, and shown in the browser display.

Data Properties and Reactivity

`data()` 是一个函数，返回一个对象，这个对象里的每个属性就是一个 **data property**，也就是组件的响应式状态数据

Both `{{ }}` and `v-bind` respond to changes in data property. This is an important feature of Vue called **reactivity**. We only need to write the template once. When some data properties are updated, Vue will update the document tree efficiently to reflect the changes.

- 响应式数据绑定 (Reactivity) : `data()` + 模板绑定
 - a. Vue 会自动把 `data()` 返回的对象变成响应式代理 (*Reactive Proxy*) 。
 - b. 当数据变化时，Vue 会自动更新 DOM 中的绑定位置。

```
JavaScript |
1 const weather = {
2   data() {
3     return {
4       temperature: 28,
5       rain: 20,
6       conditionURL: "image/sunny.png"
7     }
8   }
9 };
```

插值表达式 vs 属性绑定

类型	语法	用途	是否响应式
插值表达式	<code>{{ temperature }}</code>	显示文本内容	✅ 是
属性绑定	<code>:src="conditionURL" 或 v-bind:src</code>	设置 HTML 属性值	✅ 是

插值表达式不能用于 HTML 属性值，

比如不能写 ``

因为，HTML 的限制：HTML 属性值是字符串 => 不会执行 `{{ }}`

样式绑定：

```
HTML |
1 <span :style="styleTemperature">{{ temperature }}°C</span>
```

```
Java |
1 styleTemperature: { fontWeight: 'bold', color: 'orange' }
```

- 可以绑定整个样式对象
- Vue 会自动将对象转换为内联样式

表达式绑定：

```
1 
```

- 可以在绑定中使用三元表达式、函数调用等
- 适合根据数据动态切换属性值

Vue如何区分表达式与属性?

模板解析器会根据**语法位置**:

插值表达式	出现在 HTML 元素的内容中	<pre><p>{{ message }}</p></pre>
属性绑定表达式	出现在 HTML 属性中, 必须使用 <code>:</code> 前缀	<pre></pre>

Computed/ data/ methods

1. data() – 定义原始数据

在 Vue 中, `data()` 是一个**函数**, 它必须**返回一个对象**, 这个对象就是组件的**响应式数据源**。可以直接在模板中使用

```
1 data() {
2   return {
3     count: 0,
4   };
5 }
```

```
data() {
  return {
    temperature: 28,
    rain: 10
  };
}
```

- `data()` 是函数
- `{ temperature: 28, rain: 10 }` 是它返回的对象
- Vue 会把这个对象变成响应式代理 (Proxy), 并挂载到组件实例上
- ✅ 所以你可以在模板中写 `{{ temperature }}`, Vue 会自动追踪这个值的变化并更新界面。

2. computed – 基于data的派生状态

只读属性（不能直接赋值），适合封装复杂逻辑。不可传参

`computed` 的“参数”其实是它自动依赖的响应式数据 `data()`

`computed: { isHot() { ... } }` 是怎么调用的？

这是 Vue 的计算属性机制，它的调用方式是：

- Vue 会自动调用 `isHot()`，并把返回值作为 `vm.isHot` 的值
- 它是一个只读属性，你可以在模板或代码中直接访问它

```
1 computed: {
2   isHot() { //没有参数!!!
3     return this.temperature > 30;
4   }
5 }
```

- 在模板中你可以写： `<p>{{ isHot }}</p>`

✅ 不需要你手动调用 `isHot()`，Vue 会自动执行并缓存结果

=> 布尔值，返回true/false

3. methods

可调用、可传参。可访问 data 和 computed

`methods` 中定义的函数不会自动执行，必须由你手动调用。

JavaScript ^ 复制

```
methods: {
  greet(name) {
    alert(`Hello, ${name}`);
  }
}
```

✅ 模板中调用:

Html ^ 复制

```
<button @click="greet('Congwen')">Say Hello</button>
```

- `@click="greet('Congwen')"` 是事件绑定
- 当用户点击按钮时, Vue 才会调用 `greet()` 方法

手动传入非响应式变量的例子

JavaScript ^ 复制

```
methods: {
  checkHot(temp) {
    return temp > 30;
  }
}
```

✅ 模板调用:

Html ^ 复制

```
<button @click="checkHot(28)">Check</button>
```

如果你用 `v-model` 绑定了 `data` 中的属性

HTML |

```
1 <input v-model="temperature">
```

- `temperature` 是响应式的 (定义在 `data()` 中)
- 用户输入会自动更新 `this.temperature`
- 所有依赖 `temperature` 的 `computed`、模板、`watch` 都会自动响应

生命周期钩子 (如 `mounted`, `created`, `updated`)

生命周期钩子是 Vue 在组件运行的不同阶段自动调用的函数

你只需要定义它们，Vue 会在合适的时机触发。

<code>created</code>	实例创建后	初始化数据、调用 API、启动定时器
<code>mounted</code>	DOM 挂载后	操作 DOM、启动定时器
<code>updated</code>	响应式更新后	处理更新后的副作用
<code>unmounted</code>	组件销毁前	清理资源、移除监听器

▼ `mounted ()`

JavaScript |

```
1 ▼ mounted() {
2   console.log('组件已经挂载到 DOM');
3 }
```

- 当组件挂载到页面时，Vue 会自动调用它
- 你不需要手动执行 `mounted()`，Vue 会帮你做

this 指向

在 `computed`、`methods`、生命周期钩子中，`this` 指向组件实例

在 <code>computed</code> 中访问响应式数据	Vue 自动追踪 <code>this.xxx</code> 的依赖
在 <code>methods</code> 中访问组件状态	<code>this</code> 指向组件实例，包含 <code>data</code> ， <code>props</code> ， <code>computed</code> 等
在生命周期钩子中访问数据或调用方法	<code>this</code> 是组件上下文
在模板表达式中间接访问方法或属性	Vue 会自动绑定 <code>this</code> 到组件实例

```
Vue |
1  methods: {
2    greet(name) {
3      console.log(`Hello, ${name}`);
4    },
5    init() {
6      this.greet(this.username); // ✓ 在方法中调用另一个方法
7    }
8  },
9  mounted() {
10   this.init(); // ✓ 在生命周期钩子中调用方法
11 }
```

this 在 Vue 中指向的是组件实例

当你调用 `Vue.createApp(...).mount('#app')`，Vue 会创建一个组件实例

Vue 中，`methods` 是挂载在组件实例上的

🔍 二、如果你不加 `this` 会发生什么?

JavaScript ^ 复制

```
init() {
  greet(username); // ✗ 错误: 找不到 greet 和 username
}
```

- JavaScript 会在当前作用域查找 `greet` 和 `username`
- 但它们不是局部变量，而是组件实例上的属性
- 所以你必须用 `this.greet()` 和 `this.username`

`mounted` 是一个生命周期钩子（hook），它是组件选项中的一个独立字段，不属于 `methods`。

Vue 会在组件挂载到 DOM 后自动调用 `mounted()`，你只需要在组件选项中定义它

小结

数据定义	<code>data()</code>	定义状态
文本绑定	<code>{{ }}</code>	显示数据
属性绑定	<code>v-bind:attr</code> 或 <code>:attr</code>	动态设置属性
样式绑定	<code>:style</code>	控制样式

3-3

条件判断

Vue provides directives `v-if`, `v-else-if` and `v-else` to select what HTML content to show

写在template中!!!

```
...
<div>Temperature: {{ temperature }}°C
  <span v-if="temperature >= 25">It's hot!</span>
  <span v-else-if="temperature >= 10">pleasant weather</span>
  <span v-else>It's cold, stay warm</span>
</div>
<div>Rain: {{ rain }}%
  <img v-if="parseFloat(rain) >= 30" src='image/umbrella.png'
</div>
</div>
```

src=' ' src是固定的不会改变的, 而不是绑定的属性值

循环

v-for

```
<div id="app">
  <p v-if="sport.length === 0">I don't really like sports</p>
  <p v-else>My favorite sports:
    <span v-for='x in sport' class='box'>{{ x }}</span>
  </p>
</div>
```

写在template中!!!

v-for=""

```
<script>
const hobby = {
  data() {
    return {
      sport: [ 'football', 'running', 'swimming' ]
    }
  }
};
const vm = Vue.createApp(hobby).mount('#app');
</script>
```

3-4

异步函数调用机制

不需要手动调用，注册后由系统或事件驱动自动调用

- 用户界面事件（如点击）
- JavaScript 定时器（如 `setTimeout`）
- Vue 生命周期钩子（如 `mounted`，`updated`）

UI Event Handling

Trigger an action --> mouse click

`v-on:click` / `@click`

```
1 <button v-on:click="count++">直接执行表达式，修改响应式数据</button>
2 <button @click="this.count--">Minus 1</button>
3 <button @click="reset">调用 methods
4   中定义的函数 Reset to 0 </button>
```

options object. Inside the method body, you can use `this` to refer to data properties. Examine the view model `vm` in browser's developer tools to check that both data properties and methods are properties of `vm`.

Vue component method definition cannot use arrow function, because it requires `this`.

如何理解在 HTML 中写 JavaScript 语句？

1. 事件绑定中写 JS 表达式

```
1 <button @click="count++">Add 1</button>
```

- `count++` 是一个 JavaScript 表达式

2. 调用方法：

```
1 <button @click="reset()">Reset</button>
```

Timer

1. **One-time timer** will run a given function *once*
2. **Periodic timer** will run a given function **repeatedly** with an interval of a fixed period

- 一次性定时器:

```
1 let timerId = setTimeout(() => {
2   console.log('Time is up');
3 }, 5000);
4 clearTimeout(timerId); // 取消定时器
```

`setTimeout()` 的参数结构

```
1 setTimeout(callback, delay, ...args);
```

参数	类型	说明
<code>callback</code>	函数	要在延迟后执行的函数
<code>delay</code>	数字 (毫秒)	延迟时间 (单位: ms)
<code>...args</code>	任意	可选参数, 传给 <code>callback</code>

```
setTimeout((name) => {
  console.log('Hello, ${name}');
}, 3000, 'Congwen'); // callback function
```

Callback (回调函数) 是一个传给另一个函数的函数, 它不会立即执行, 而是在未来某个时刻被“回调”执行。

补充: 函数的参数

函数的参数就是为了在函数体内使用而定义的变量。它们是函数的输入值

Html ^ 复制

```
<button @click="sayHello">Click Me</button>
```

Javascript ^ 复制

```
methods: {  
  sayHello() {  
    alert('Hello!');  
  }  
}
```

- `sayHello` 是你定义的函数
- Vue 会在用户点击按钮时 自动“回调”这个函数

- 周期性定时器:

JavaScript |

```
1 let timerId = setInterval(() => {  
2   console.log('2s passed');  
3 }, 2000);  
4 clearInterval(timerId); // 取消周期性定时器
```

`let timerId` 是一个变量，用来保存定时器的 ID，以便你后续可以取消它。

Lifecycle Methods

By defining a **lifecycle hook** in the options object, we can instruct Vue what to do in different moments in the lifecycle.

```
<div id="app">  
  <p>This component is created {{ count }} seconds ago.</p>  
</div>  
  
<script>  
const opts = {  
  data() {  
    return { count: 0 };  
  },  
  created() {  
    setInterval(() => { this.count++ }, 1000);  
  },  
}
```

3-5.

Bi-directional Binding with `v-model`

双向绑定 --> 数据属性和表单控件的值保持同步

When a **user inputs or changes text** in the **input control**, the **data property** is updated immediately.

Similar, when the **data property** **name** **changes**, the **input control** also refreshes with the update.

```
<div id="app">
  <div>Name:</div>
  <input v-model="name">
  Hello {{ name }}!
</div>

<script>
const opts = {
  data() {
    return { name: '' }
  }
};
Vue.createApp(opts).mount('#app');
</script>
```

用户输入 → 数据	输入框变化	DOM 事件监听	更新 <code>data</code> 中的值
数据 → 用户界面	数据变化	响应式系统	更新输入框和绑定区域

V-model 主要用于处理用户输入，并实现数据的双向绑定

用户输入 → 自动更新数据

数据变化 → 自动更新界面

`v-model` 让你不需要手动监听 `input` 或 `change` 事件，也不需要手动更新数据。

`v-model` 的作用是把用户输入或选择的值绑定到组件的数据属性中，而你是否显示它是视图层的选择，并不影响你在逻辑层使用它。

文本输入框	<code><input v-model=""></code>
多行文本框	<code><textarea v-model=""></code>
复选框	<code><input type="checkbox" v-model=""></code>
单选框	<code><input type="radio" v-model="" value="A"></code>

下拉菜单

```
<select v-model="">
```